

**PROFESSIONAL CERTIFICATE
IN MACHINE LEARNING AND
ARTIFICIAL INTELLIGENCE**

**Office Hour #23 with
Matilde D'Amelio**

Convolutional Neural Networks

What do CNN layers learn?

Each CNN layer learns filters of increasing complexity.

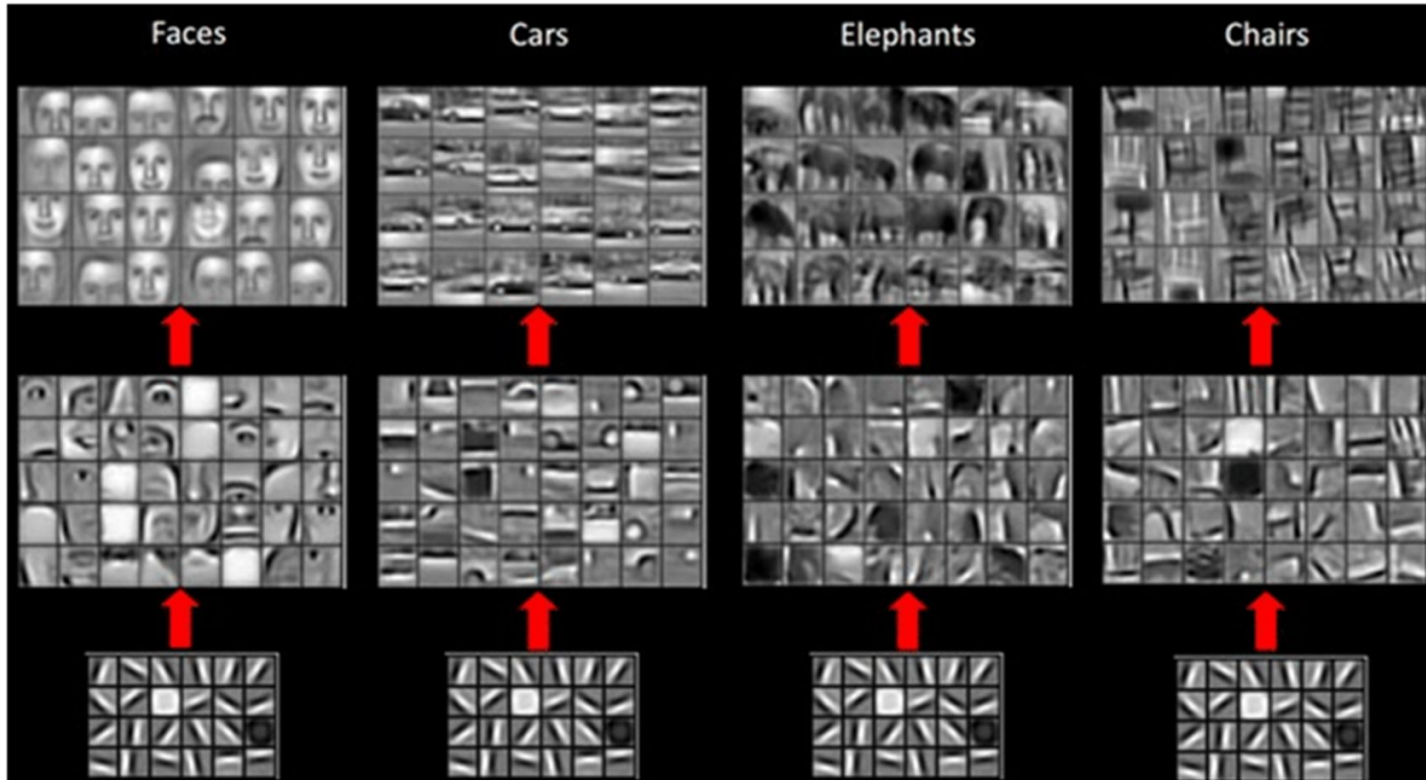
The first layers learn **basic feature detection filters**: edges, corners, etc.

The middle layers learn filters that detect **parts of objects**. For faces, they might learn to respond to eyes, noses, etc.

The last layers have higher representations: they learn to **recognize full objects**, in different shapes and positions.

Convolutional Neural Networks

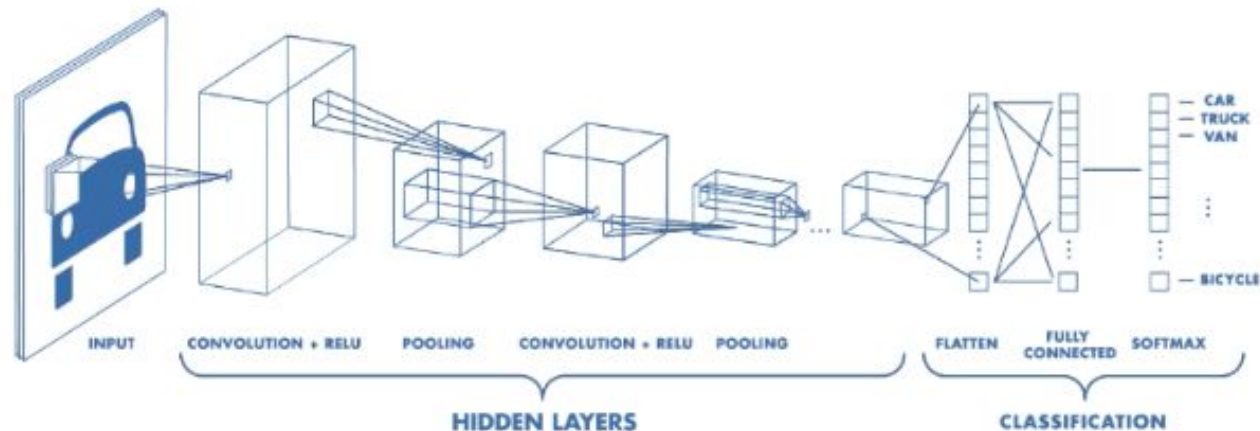
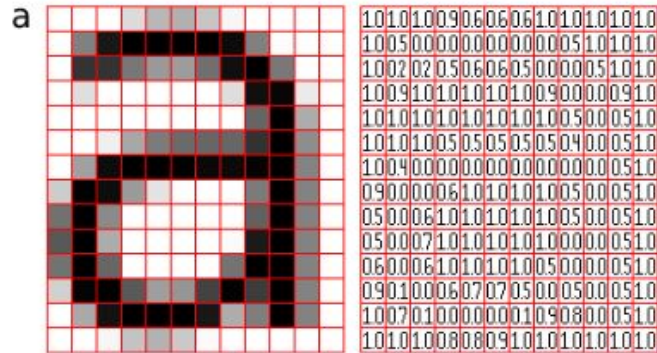
Fully Built CNN (VGG)



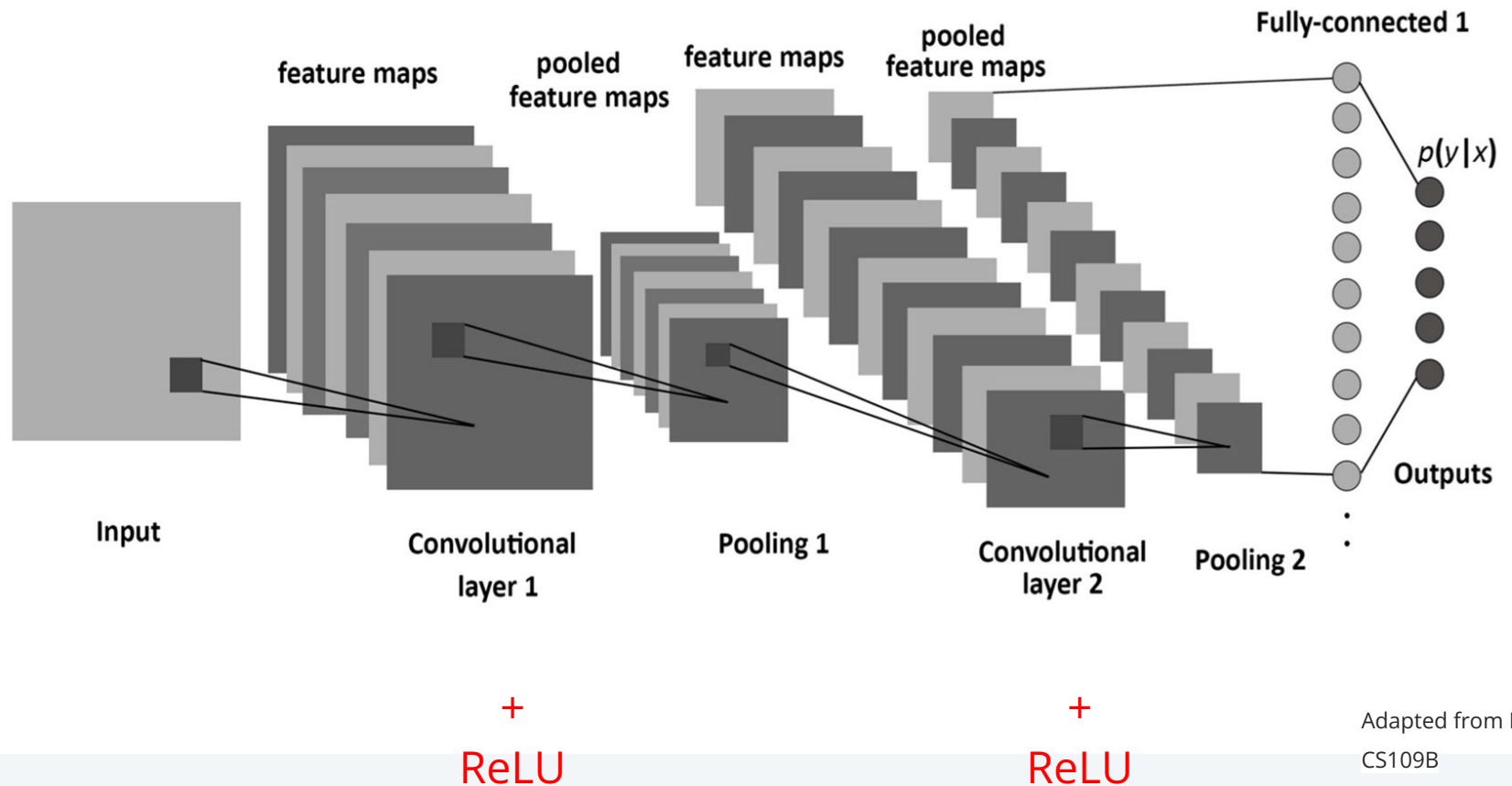
Convolutional Neural Network

A Convolutional Neural Network, process data that has a grid-like topology, such as an image. A digital image is a binary representation of visual data. It contains a series of pixels arranged in a grid-like fashion that contains pixel values to denote how bright and what color each pixel should be

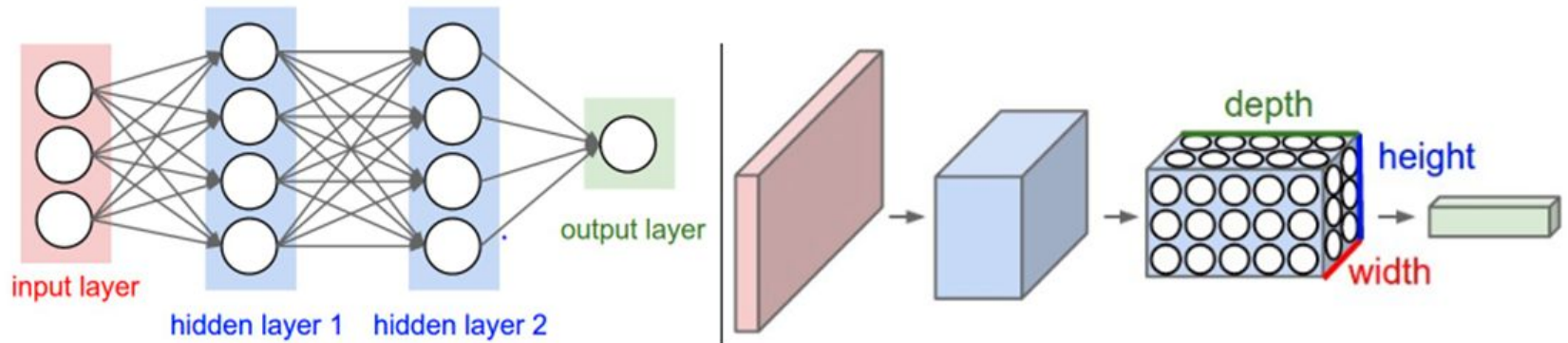
A CNN typically has three layers: **a convolutional layer, a pooling layer, and a fully connected layer.**



Convolutional Neural Networks



Convolutional Neural Networks



CNNs are also composed of layers, but those layers are not fully connected: they have **filters**, sets of cube-shaped weights that are applied throughout the image. Each 2D slice of the filters are called **kernels**.

These filters introduce **translation invariance** and **parameter sharing**.

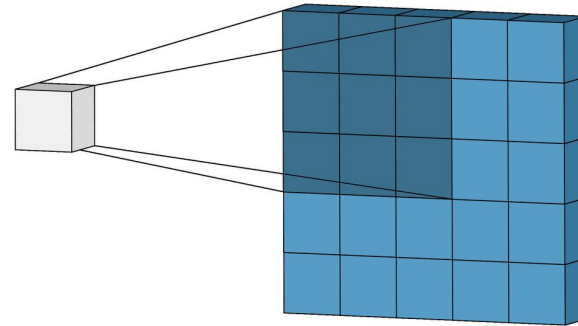
How are they applied? Convolutions!

Convolutional Neural Network

Convolution Layer

The convolution layer is the core building block of the CNN. It carries the main portion of the network's computational load.

This layer performs a dot product between two matrices, where one matrix is the set of learnable parameters otherwise known as a **kernel**, and the other matrix is the restricted portion of the receptive field. **The kernel is spatially smaller than an image but is more in-depth. This means that, if the image is composed of three (RGB) channels, the kernel height and width will be spatially small, but the depth extends up to all three channels.**



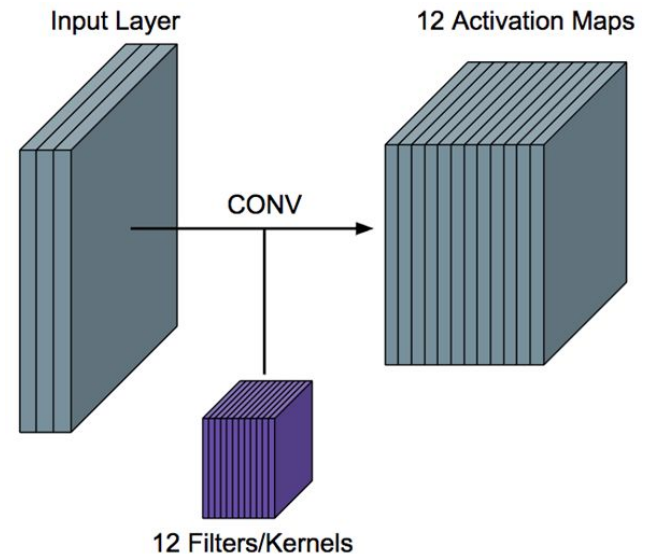
During the forward pass, the kernel slides across the height and width of the image-producing the image representation of that receptive region. This produces a two-dimensional representation of the image known as an **activation map** that gives the response of the kernel at each spatial position of the image. The sliding size of the kernel is called a stride.

Convolutional Neural Networks

To be clear: each filter is convolved with the entirety of the **3D input cube**, but generates a **2D feature map**.

Because we have multiple filters, we end up with a 3D output: **one 2D feature map per filter**.

The feature map dimension can **change drastically** from one conv layer to the next: we can enter a layer with a 32x32x16 input and exit with a 32x32x128 output if that layer has 128 filters.



Convolutional Neural Networks

A convolutional layer convolves each of its filters with the input.

Input: a **3D tensor**, where the dimensions are Width, Height and Channels (or Feature Maps)

Output: a **3D tensor**, with dimensions Width, Height and Feature Maps (one for each filter)

Applies non-linear **activation function** (usually ReLU) over each value of the output.

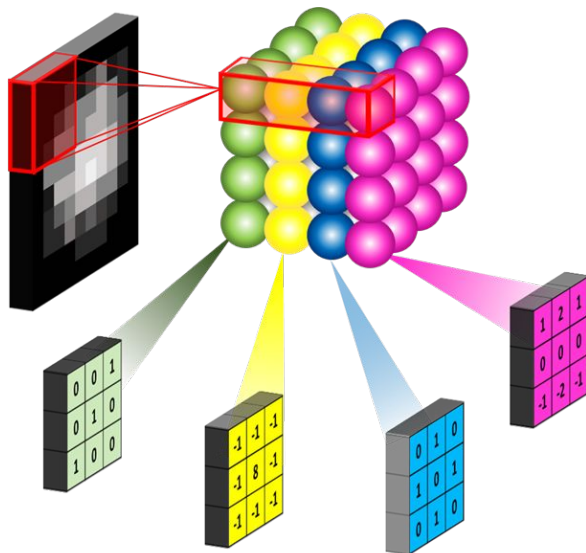
Multiple **parameters to define**: number of filters, size of filters, stride, padding, activation function to use, regularization.

Convolutional Neural Networks

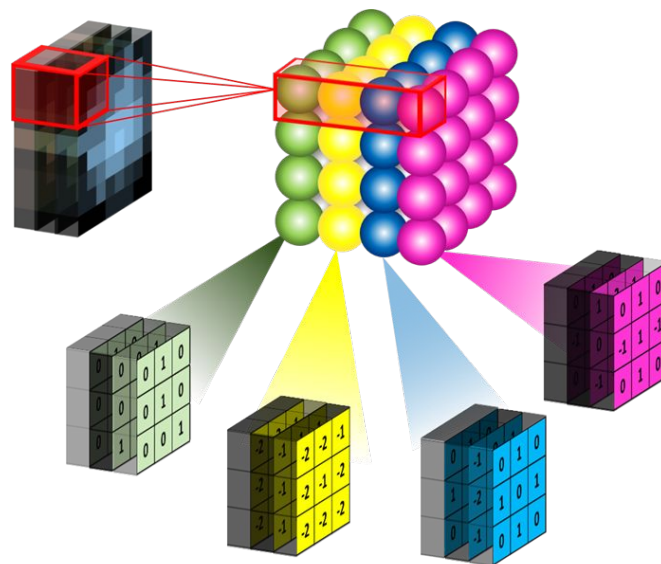


Input

CNN



Convolutional layer with four 3x3 filters on a **black and white image** (just one channel)

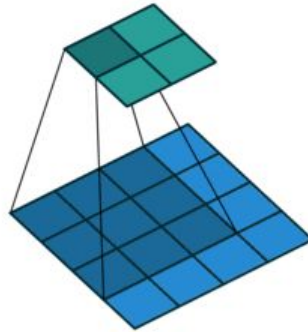


Convolutional layer with four 3x3 filters on an **RGB image**. As you can see, the filters are now cubes, and they are applied on the full depth of the image..

Convolutional Neural Networks

Padding

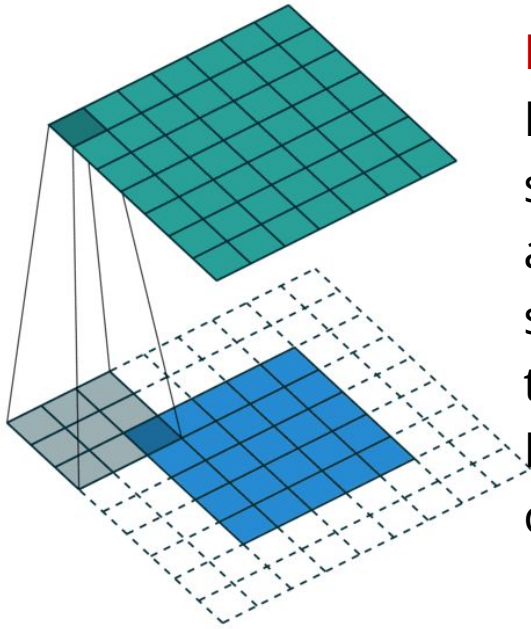
If we apply convolutions on a normal image, the result will be down-sampled by an amount depending on the size of the filter.



We can avoid this by padding the edges in different ways.

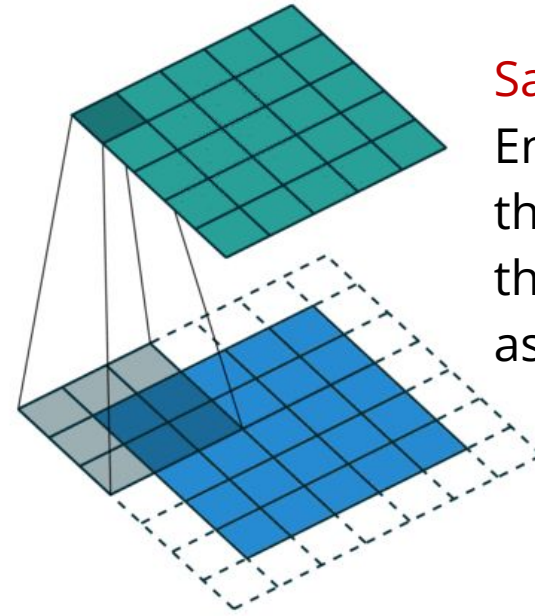
Convolutional Neural Networks

Padding



Full padding.

Introduces zeros such that all pixels are visited the same amount of times by the filter. Increases size of output.



Same padding.

Ensures that the output has the same size as the input.

Convolutional Neural Networks

Stride

Stride controls how the filter convolves around the input volume.

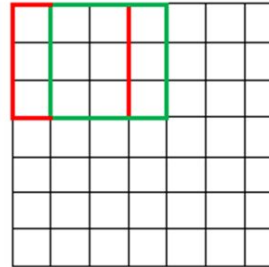
The formula for calculating the output size is:

$$O = (W - K + 2P) / S + 1$$

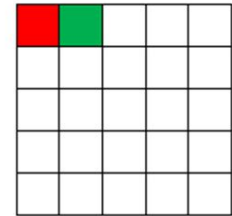
Where O is output dim, W is the input dim, K is the filter size, P is padding and S the stride

Stride 1

7 x 7 Input Volume

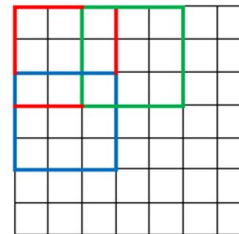


5 x 5 Output Volume

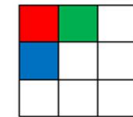


Stride 2

7 x 7 Input Volume



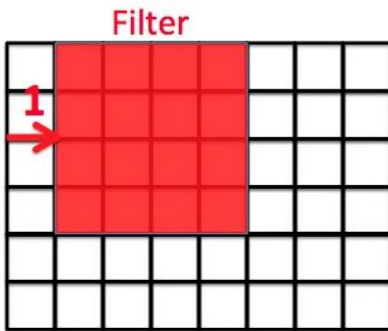
3 x 3 Output Volume



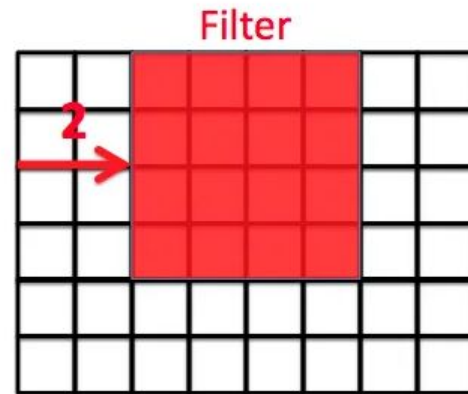
Convolutional Neural Networks

Stride

If stride = 1, the filter will move one pixel.



If stride = 2, the filter will move two pixels.



Convolutional Neural Networks

In a convolutional layer, we are basically applying multiple filters at over the image to extract different features.

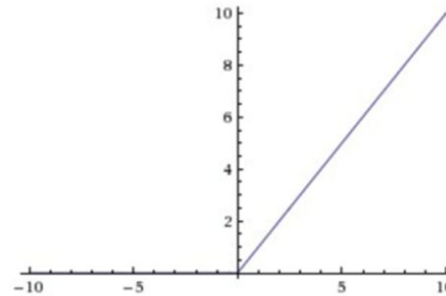
But most importantly, **we are learning those filters!**

One thing we're missing: non-linearity.

Convolutional Neural Networks

The most successful non-linearity for CNNs is the Rectified Non-Linear unit (ReLU):

$$\text{Output} = \text{Max}(\text{zero}, \text{Input})$$



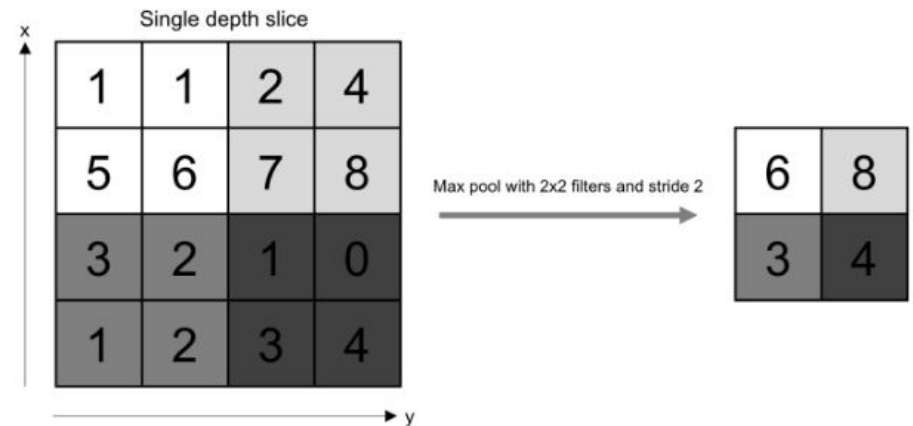
Combats the vanishing gradient problem occurring in sigmoids, is easier to compute, generates sparsity (not always beneficial)

Convolutional Neural Network

Pooling Layer

The pooling layer replaces the output of the network at certain locations by deriving a summary statistic of the nearby outputs. This helps in **reducing the spatial size of the representation**, which decreases the required amount of computation and weights. The pooling operation is processed on **every slice** of the representation individually.

There are several pooling functions such as the average of the rectangular neighborhood, L2 norm of the rectangular neighborhood, and a weighted average based on the distance from the central pixel. However, the most popular process is **max pooling**, which reports the maximum output from the neighborhood.



Convolutional Neural Networks

Pooling

Invariant to small, “local transitions”

–Face detection: enough to check the presence of eyes, not their precise location

Reduces input size to final fully connected layers

No learnable parameters

1	1	2	5
5	7	7	8
3	1	1	0
1	2	3	4

→
max pool with 2x2 window
and stride 1

7	7	8

Convolutional Neural Networks

Pooling

1	1	2	5
5	7	7	8
3	1	1	0
1	2	3	4

max pool with 2x2 window
and stride 2



7	8
3	4

1	1	2	5
5	7	7	8
3	1	1	0
1	2	3	4

mean pool with 2x2 window
and stride 2



7	11
3.5	4

Convolutional Neural Network

Fully Connected Layer

Neurons in this layer have full connectivity with all neurons in the preceding and succeeding layer. This is why it can be computed as usual by a matrix multiplication followed by a bias effect.

There are several fully connected layers:

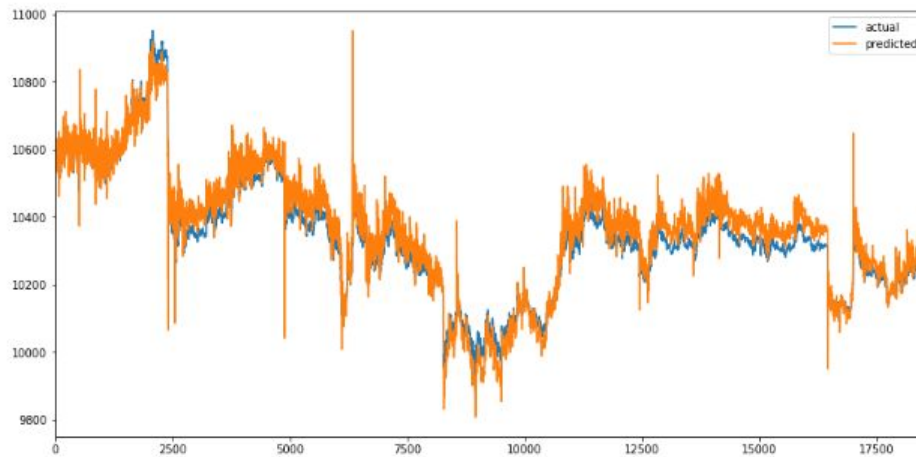
- **Fully connected input layer:** This is also known as the flattening of the images. The outputs gained from the last layer are flattened into a single vector so that it can be used as the input data from the upcoming layer.
- **Fully connected layer:** After the feature analysis has been done and it's time for computation, this layer assigns random weights to the inputs and predicts a suitable label.
- **Fully connected Output layer:** This is the final layer of the CNN model which contains the results of the labels determined for the classification and assigns a class to the images.

Convolutional Neural Network: Application

1. **Object detection:** With CNN, we now have sophisticated models like [R-CNN](#), [Fast R-CNN](#), and [Faster R-CNN](#) that are the predominant pipeline for many object detection models deployed in autonomous vehicles, facial detection, and more.
2. **Semantic segmentation:** In 2015, a group of researchers from Hong Kong developed a CNN-based [Deep Parsing Network](#) to incorporate rich information into an image segmentation model. Researchers from UC Berkeley also built [fully convolutional networks](#) that improved upon state-of-the-art semantic segmentation.
3. **Image captioning:** CNNs are used with recurrent neural networks to write captions for images and videos. This can be used for many applications such as activity recognition or describing videos and images for the visually impaired. It has been heavily deployed by YouTube to make sense to the huge number of videos uploaded to the platform on a regular basis.

Long Short Term Memory (for Time Series)

Eg: train a Long Short Term Memory Neural Network (LSTM) on Bitcoin trading data and use it to predict the price of unseen trading data.



Actual and predicted VWAP on the test set

QUESTIONS?

